

Para implementar este diseño arquitectónico de alta concurrencia (**EDA + CQRS + Actor Model**) y validar las hipótesis mediante el experimento de **Stress Test (Fase 1 y Fase 2)**, es necesario reestructurar los componentes de software en Python para reflejar con precisión los patrones de diseño, tácticas y métricas requeridas.

A continuación, se detalla la reingeniería completa del ecosistema para tu despliegue en Docker Desktop.

1. Estructura del Proyecto Completa

Asegúrate de renombrar tus archivos eliminando cualquier extensión .yaml tal como lo resolvimos anteriormente:

```
|— docker-compose.yml
|— api_gateway/
|   |— Dockerfile
|   └─ gateway.py
|— matching_engine/
|   |— Dockerfile
|   └─ engine.py
└─ injector/
    |— Dockerfile
    └─ locustfile.py
```

2. docker-compose.yml (Infraestructura y Red Asíncrona)

Se añade el **API Gateway** como el componente ligero de ingesta (Command Flow), implementando límites estrictos y políticas de red eficientes.

```
services:
  # Nodo 2: Queue Broker (Buffer Asíncrono - RabbitMQ)
  queue-broker:
    image: rabbitmq:3-management
    container_name: queue-broker
    ports:
      - "5672:5672"
      - "15672:15672"
    deploy:
      resources:
```

```

        limits:
            cpus: '2.0'
            memory: 4096M
    healthcheck:
        test: ["CMD", "rabbitmq-diagnostics", "check_running"]
        interval: 5s
        timeout: 3s
        retries: 5

```

Ingesta: API Gateway (Simulador HTTP Ligero) - CQRS Command Flow

```

api-gateway:
    build: ./api_gateway
    container_name: api-gateway
    ports:
        - "8000:8000"
    depends_on:
        queue-broker:
            condition: service_healthy
    deploy:
        resources:
            limits:
                cpus: '2.0'
                memory: 2048M
    environment:
        - RABBITMQ_HOST=queue-broker

```

Nodo 3: Matching Engine (Actor Model & In-Memory Processing)

```

matching-engine:
    build: ./matching_engine
    container_name: matching-engine
    depends_on:
        queue-broker:
            condition: service_healthy
    deploy:
        resources:
            limits:
                cpus: '4.0'
                memory: 8192M
    environment:
        - RABBITMQ_HOST=queue-broker
        - PREFETCH_COUNT=100
    ulimits:
        memlock:
            soft: -1
            hard: -1

```

Nodo 1: Inyector de Carga Extrínseco (Locust con perfiles de

```
Stress Test)
injector:
  build: ./injector
  container_name: injector
  ports:
    - "8089:8089"
  depends_on:
    api-gateway:
      condition: service_started
  deploy:
    resources:
      limits:
        cpus: '2.0'
        memory: 4096M
  environment:
    - TARGET_URL=http://api-gateway:8000
```

3. Componente API Gateway (api_gateway/)

Este endpoint ligero en Python implementa el desacoplamiento inmediato. Recibe el comando de compra/venta, estampa el tiempo inicial de ingesta (ts_ingest_start), despacha asíncronamente al buffer y retorna un ACK con latencia sub-milisegundo.

- **api_gateway/Dockerfile:**

```
FROM python:3.12-slim
WORKDIR /app
RUN pip install --no-cache-dir fastapi uvicorn pika
COPY gateway.py .
CMD ["uvicorn", "gateway.py:app", "--host", "0.0.0.0", "--port",
"8000", "--workers", "4"]
```

- **api_gateway/gateway.py:**

```
from fastapi import FastAPI, HTTPException, Response
from pydantic import BaseModel
import pika
import json
import time
import os

app = FastAPI()
RABBITMQ_HOST = os.getenv('RABBITMQ_HOST', 'localhost')

# Inicialización del canal de mensajería persistente con límite de
profundidad opcional
connection =
```

```

pika.BlockingConnection(pika.ConnectionParameters(host=RABBITMQ_HOST))
channel = connection.channel()
channel.queue_declare(
    queue='orders_buffer',
    durable=True,
    arguments={
        'x-max-length': 500000, # Límite de profundidad máxima del
Buffer
        'x-overflow': 'reject-publish' # Política ante saturación
(Fase 2)
    }
)

```

```

class OrderSchema(BaseModel):
    order_id: str
    trader_id: str
    symbol: str
    side: str
    price: float
    quantity: int
    timestamp: int

```

```

@app.post("/api/v1/orders")
async def ingest_order(order: OrderSchema):
    ts_ingest_start = time.time_ns() # Captura en nanosegundos

    payload = order.model_dump()
    payload['ts_ingest_start'] = ts_ingest_start

    try:
        channel.basic_publish(
            exchange='',
            routing_key='orders_buffer',
            body=json.dumps(payload),
            properties=pika.BasicProperties(delivery_mode=2)
        )
        # Medición de Latencia de Ingesta (Simulada para respuesta
rápida)
        ts_ingest_end = time.time_ns()
        latency_ms = (ts_ingest_end - ts_ingest_start) /
1_000_000.0

        return {"status": "ACK", "ingest_latency_ms": latency_ms}
    except Exception as e:
        raise HTTPException(status_code=503, detail="Buffer
Saturated or Unreachable")

```

4. Componente Matching Engine Core (matching_engine/)

Implementa el **Modelo de Actores** aislando los libros de órdenes por símbolo en sub-hilos lógicos coordinados en memoria (heapq), controlando el flujo mediante **Prefetch Count** y simulando la persistencia en disco diferida (**Write-Behind**) mediante lotes (Batching).

- **matching_engine/Dockerfile:**

```
FROM python:3.12-slim
WORKDIR /app
RUN pip install --no-cache-dir pika
COPY engine.py .
CMD ["python", "engine.py"]
```

- **matching_engine/engine.py:**

```
import pika
import json
import heapq
import os
import time
import threading

RABBITMQ_HOST = os.getenv('RABBITMQ_HOST', 'localhost')
PREFETCH_COUNT = int(os.getenv('PREFETCH_COUNT', '100'))

# Estructura del Modelo de Actores: Un hilo/estructura limpia por
# cada símbolo (Libro aislado sin bloqueos globales)
class OrderBookActor:
    def __init__(self, symbol):
        self.symbol = symbol
        self.buys = [] # Max-Heap (precios invertidos)
        self.sells = [] # Min-Heap
        self.batch_persistence = []
        self.lock = threading.Lock()

    def process_order(self, order):
        with self.lock:
            ts_match_start = time.time_ns()
            side = order['side']
            price = float(order['price'])
            qty = int(order['quantity'])

            matches_executed = 0

            if side == 'BUY':
                # Intentar emparejar contra las órdenes de venta
```

```

más baratas
        while self.sells and self.sells[0][0] <= price and
qty > 0:
            sell_price, sell_order =
heapq.heappop(self.sells)
            match_qty = min(qty, sell_order['quantity'])
            qty -= match_qty
            sell_order['quantity'] -= match_qty

            if sell_order['quantity'] > 0:
                heapq.heappush(self.sells, (sell_price,
sell_order))

            matches_executed += 1
            self._trigger_write_behind(order, sell_order,
match_qty)

        if qty > 0:
            heapq.heappush(self.buys, (-price, order))
        else:
            # Intentar emparejar contra las órdenes de compra
más caras
        while self.buys and -self.buys[0][0] >= price and
qty > 0:
            neg_buy_price, buy_order =
heapq.heappop(self.buys)
            match_qty = min(qty, buy_order['quantity'])
            qty -= match_qty
            buy_order['quantity'] -= match_qty

            if buy_order['quantity'] > 0:
                heapq.heappush(self.buys, (neg_buy_price,
buy_order))

            matches_executed += 1
            self._trigger_write_behind(buy_order, order,
match_qty)

        if qty > 0:
            heapq.heappush(self.sells, (price, order))

        ts_match_end = time.time_ns()

        # Métricas Clave de Latencia Interna
        core_latency_ms = (ts_match_end - ts_match_start) /
1_000_000.0
        total_pipeline_latency_ms = (ts_match_end -
order['ts_ingest_start']) / 1_000_000.0

```

```

        print(f"[ENGINE] Símbolo: {self.symbol} | Match:
{matches_executed} | Latencia Core: {core_latency_ms:.2f}ms |
Latencia End-to-End: {total_pipeline_latency_ms:.2f}ms")

    def _trigger_write_behind(self, buy_order, sell_order, qty):
        # Táctica: Escritura Asíncrona Diferida por Lotes
        (Batching)
        match_event = {"buy_id": buy_order['order_id'], "sell_id":
sell_order['order_id'], "quantity": qty, "ts": time.time()}
        self.batch_persistence.append(match_event)

        if len(self.batch_persistence) >= 50: # Tamaño de Lote
            # Simulación de volcado asíncrono a disco sin bloquear
            la ruta crítica en RAM
            # En producción real aquí iría un hilo secundario
            guardando a PostgreSQL/Mongo
            self.batch_persistence.clear()

# Catálogo dinámico de Actores
actors = {}

def get_actor(symbol) -> OrderBookActor:
    if symbol not in actors:
        actors[symbol] = OrderBookActor(symbol)
    return actors[symbol]

def on_message_received(ch, method, properties, body):
    order = json.loads(body)
    actor = get_actor(order['symbol'])
    actor.process_order(order)
    ch.basic_ack(delivery_tag=method.delivery_tag)

# Configuración del Consumidor
connection =
pika.BlockingConnection(pika.ConnectionParameters(host=RABBITMQ_HO
ST))
channel = connection.channel()
channel.queue_declare(queue='orders_buffer', durable=True)

# Táctica: Controlar la saturación mediante Tasa de Prefetch
channel.basic_qos(prefetch_count=PREFETCH_COUNT)
channel.basic_consume(queue='orders_buffer',
on_message_callback=on_message_received)

print(f"[*] Engine Conectado. Prefetch Count: {PREFETCH_COUNT}.
Procesando en memoria...")
channel.start_consuming()

```

5. Inyector de Carga Extrínseco (injector/)

Locust se configura con las tasas exactas especificadas para simular adecuadamente la **Fase 1** (Línea Base: 1,300 req/min) y la **Fase 2** (Pico de volatilidad: 6,500 req/min).

- **injector/Dockerfile:**

```
FROM python:3.12-slim
WORKDIR /app
RUN pip install --no-cache-dir locust
COPY locustfile.py .
# Inicia la interfaz web en el puerto 8089
CMD ["locust", "-f", "locustfile.py"]
```

- **injector/locustfile.py:**

```
from locust import HttpUser, task, between
import uuid
import random
import time
import os

class FinancialTrader(HttpUser):
    wait_time = between(0.01, 0.05) # Permite ejecutar alta
    concurrencia por segundo

    def on_start(self):
        self.symbols = ['AAPL', 'MSFT', 'GOOGL', 'AMZN', 'TSLA']
        self.trader_id = f"TRADER_{random.randint(1, 1000)}"

    @task
    def post_order(self):
        # Determinación de lado basada en cuotas (800 BUYS vs 500
        SELLS de Fase 1)
        side = 'BUY' if random.random() < (800 / 1300) else 'SELL'

        payload = {
            "order_id": str(uuid.uuid4()),
            "trader_id": self.trader_id,
            "symbol": random.choice(self.symbols),
            "side": side,
            "price": round(random.uniform(100.0, 1500.0), 2),
            "quantity": random.randint(10, 200),
            "timestamp": int(time.time() * 1000)
        }
```



```
headers = {'Content-Type': 'application/json'}

# Envío directo al CQRS Ingestion Endpoint (API Gateway)
self.client.post("/api/v1/orders", json=payload,
headers=headers)
```



Guía para Validar el Experimento de Estrés

Paso 1: Lanzar el Entorno Estructurado

Ejecuta la construcción en la raíz del proyecto:

```
docker compose up --build
```

Paso 2: Ejecución de la Fase 1 (Línea Base - 10 Minutos)

1. Abre tu navegador e ingresa a la interfaz de Locust: <http://localhost:8089>
2. Configura los parámetros para alcanzar **1,300 órdenes/minuto** (~22 órdenes por segundo):
 - **Number of users:** 40
 - **Spawn rate:** 2
 - **Host:** <http://api-gateway:8000>
3. Monitorea las métricas en Locust y el log de matching-engine. El p95 del Core de emparejamiento debe mantenerse ≤ 200 ms.

Paso 3: Ejecución de la Fase 2 (Pico de Volatilidad - 30 Minutos)

1. Sin detener la infraestructura, incrementa de golpe los usuarios en la interfaz de Locust para buscar **6,500 órdenes/minuto** (~108 órdenes por segundo):
 - Cambia los usuarios activos (**Number of users**) a 250.
2. **Qué observar:**
 - La cola en RabbitMQ (<http://localhost:15672>) comenzará a acumular mensajes debido a la **Amortiguación Asíncrona**.
 - Gracias al desacoplamiento, la **Latencia de Ingesta** en la API se mantendrá baja (≤ 0.3s), aislando al Trader de la ralentización del Matching Engine.
 - Al terminar los 30 minutos, apaga el tráfico en Locust y mide el **Tiempo de Drenado** (cuánto tarda el engine en vaciar la cola acumulada a su capacidad óptima).

Si necesitas mayor precisión analítica, indícame si deseas:

- Implementar un script externo en Python que grafique el **p95 de latencias y el drenado del buffer** de manera automatizada al finalizar las fases.

- Migrar la lógica del buffer de **RabbitMQ** a **Apache Kafka** para evaluar el procesamiento por particiones distribuidas por símbolo.